

Delevrili - Master Product and Technical Specification

Version: 1.0

Date: 2026-03-13

Status: Ready for Development Handoff

1. Document Purpose

This file is the single source of truth for building **Delevrili**.

It replaces and consolidates all previous separate docs (brief, PRD, architecture, epics, UX notes). A development team should be able to implement MVP from this file only.

2. Project Identity

2.1 Product Name

Delevrili

2.2 What Delevrili Is

Delevrili is a delivery marketplace for Algeria that connects: - **Shippers** (businesses that need goods delivered) - **Couriers** (independent drivers/riders who deliver)

The platform digitizes dispatching and replaces phone-call coordination with structured posting, bidding, assignment, tracking, proof of delivery, and ratings.

2.3 Core Value

- For shippers: post once, receive bids fast, track every delivery.
- For couriers: find paid delivery jobs near route/home city and build a portable reputation.

3. Users and Role Boundaries

This section explicitly defines what users can and cannot do.

3.1 Roles

- **Shipper**: company dispatcher/logistics contact.
- **Courier**: independent delivery partner.
- **Admin**: Delevrili operations team.

3.2 Permissions Matrix (Can / Cannot)

Shipper Can: - Create and manage shipper account/profile. - Upload company verification documents. - Post delivery jobs with pricing and constraints. - View incoming bids. - Accept, reject, or counter bids. - Track assigned jobs and courier progress. - Confirm delivery completion. - Rate couriers after completed jobs. - View spending and job history reports.

Cannot: - Bid on jobs. - Edit courier legal documents. - View other shippers' private jobs. - Change platform commission rules. - Delete audit history.

Courier Can: - Create courier account/profile. - Upload personal/vehicle verification documents. - Browse open jobs that match profile and vehicle type. - Place bids and respond to counter-offers. - Confirm pickup and delivery milestones. - Upload proof of delivery (POD) image. - Share periodic GPS updates during active job. - Rate shippers after completed jobs. - View personal earnings and job history.

Cannot: - Post jobs. - Access another courier's earnings or documents. - Edit shipper job once assigned (except allowed status actions). - View admin dashboards or internal dispute notes. - Force-close a job.

Admin Can: - Verify/reject shipper and courier documents. - Suspend/reactivate accounts. - View platform-wide operations dashboard. - Manage disputes and ticket lifecycle. - View full audit logs and commission records. - Configure operational constants (within approved limits).

Cannot: - Alter historical financial records without audit trace. - Remove completed transaction audit trail. - Impersonate users without explicit support workflow and logged reason.

4. Business Model and Rules

4.1 Commission Model (MVP)

- Total commission: **3%** per completed job.
- Courier fee: **1.5%**.
- Shipper fee: **1.5%**.

4.2 Pricing Visibility Rule

Before bid confirmation, both sides must see a transparent breakdown: - Net courier amount. - Courier fee. - Shipper fee. - Final shipper total.

4.3 Payment Rule (MVP)

- MVP uses honor-system tracking (no built-in wallet/escrow).
- Platform records obligations and generates invoice records.

4.4 Liability Rule

- Delevrili is a marketplace facilitator.
- Cargo liability remains between shipper and courier according to their legal agreement.

5. Product Scope

5.1 In Scope (MVP)

- Onboarding and verification for shipper/courier.
- Job posting, discovery, bidding, countering, assignment.
- Job lifecycle status management.
- Active-job tracking with periodic GPS updates.
- In-app trip/job chat.
- POD upload on delivery.
- Bilateral ratings.
- Courier earnings dashboard.
- Shipper reporting dashboard.
- Admin dashboard for verification, disputes, and commission tracking.
- Push notifications.

5.2 Out of Scope (Post-MVP)

- In-app payment gateway and escrow.
- Multi-stop route optimization.
- iOS-first advanced feature parity beyond core.
- AI fraud scoring.
- Automated legal contract generation.
- Cross-border logistics.

6. End-to-End User Flows

6.1 Onboarding and Verification

1. User registers as shipper or courier.
2. User uploads required documents.
3. Account status is `PENDING_VERIFICATION`.
4. Admin reviews docs and sets `VERIFIED` or `REJECTED` with reason.
5. Verified users unlock full capabilities.

6.2 Job Posting to Assignment

1. Shipper creates job with route, cargo, timing, vehicle type, bid deadline.
2. System broadcasts job to matching verified couriers.
3. Couriers submit bids.
4. Shipper accepts/rejects/counters.
5. Accepted bid creates active assignment and auto-closes competing bids.

6.3 Execution to Close

1. Courier confirms pickup.
2. GPS updates are sent every 30 minutes while active.
3. Shipper and courier communicate in trip chat.
4. Courier confirms delivery and uploads POD photo.
5. Shipper confirms completion.
6. Both rate each other.
7. Job enters CLOSED and commission record is finalized.

7. Functional Requirements

FR-01 Identity and Access

- Role-based authentication and authorization.
- Session/token management with refresh mechanism.
- Verification-state-based feature gating.

FR-02 Shipper Job Management

- Create, edit, publish, cancel open jobs.
- Structured fields for route, cargo, timing, and vehicle requirement.
- Bid deadline and status tracking.

FR-03 Courier Discovery and Bidding

- Match open jobs by location and vehicle type.
- Bid submission with clear commission math.
- Counter-offer response and expiration logic.

FR-04 Assignment and Lifecycle

- Status model:
 - DRAFT
 - OPEN
 - ASSIGNED
 - PICKUP_CONFIRMED
 - IN_TRANSIT
 - DELIVERED
 - CLOSED
 - CANCELLED
- State transitions must be validated server-side.

FR-05 Delivery Proof and Communication

- POD image upload required for DELIVERED transition.
- Job-scoped chat available only while job is active.

FR-06 Ratings and Trust

- 1-5 star bilateral rating after completion.
- Public score visible after minimum threshold (e.g., 3 ratings).

FR-07 Operations and Admin

- Document verification queue.
- Dispute and ticket management.
- Commission and invoice tracking.
- User moderation actions with audit logs.

FR-08 Analytics and Reports

- Courier: earnings summary and completed jobs.
- Shipper: spending, route history, courier usage.
- Admin: platform KPIs and status monitoring.

8. Data Model (Core MVP)

All money values stored in integer minor units (no floating point for financials).

8.1 Core Entities

- users
- shipper_profiles
- courier_profiles
- verification_documents
- jobs
- bids
- assignments
- job_status_events
- gps_pings
- pod_files
- chat_threads
- chat_messages
- ratings
- commission_records
- invoice_records
- dispute_tickets
- audit_logs
- notifications

8.2 Entity Notes

- users: identity, role, account status, language preference.
- shipper_profiles: company metadata and legal identifiers.

- **courier_profiles**: vehicle type, home city, operational radius.
- **jobs**: route, cargo details, deadlines, required vehicle type, owner shipper.
- **bids**: bid amount, status, expiration, counter linkage.
- **assignments**: winning bid and active execution state.
- **job_status_events**: immutable timeline of state changes.
- **gps_pings**: periodic location records for active assignment.
- **pod_files**: delivery evidence metadata + storage pointer.
- **ratings**: bilateral post-delivery feedback.
- **commission_records**: per-job fee calculation results.
- **audit_logs**: admin/security-sensitive actions.

8.3 Required Indexing

- Jobs by status + pickup date.
- Bids by job_id + created_at.
- Assignments by courier_id + active status.
- GPS pings by assignment_id + timestamp.
- Documents by user_id + verification_status.
- Audit logs by actor_id + timestamp.

9. API and System Design

9.1 Architecture Style

- Client apps (web + mobile) communicate with backend API.
- API follows REST for core business workflows.
- WebSocket channel used for near-real-time chat and bid updates.

9.2 Apps

- **Web App**: shipper workspace + admin dashboard.
- **Mobile App**: courier app (Android-first MVP).
- **Backend API**: authentication, business logic, workflow orchestration.

9.3 Required Services

- Authentication/authorization service.
- Job and bidding service.
- Assignment/lifecycle service.
- Chat service.
- Notification service.
- Verification/admin service.
- Reporting service.

9.4 Storage Services

- Relational DB for transactional records.

- Object storage for docs/POD media.
- Cache layer for sessions, throttling, transient counters.

10. Tech Stack (Keep Only Needed Core Tech)

This is intentionally minimal and aligned with the original project direction.

10.1 Required Stack

- **Backend:** NestJS + TypeScript
- **Web:** Next.js + TypeScript
- **Mobile:** Expo React Native (Android MVP)
- **Database:** PostgreSQL
- **ORM:** Prisma
- **Cache/Queue support:** Redis
- **File storage:** S3-compatible object storage
- **Push notifications:** FCM

10.2 Why This Stack

- Same technology family across all apps (TypeScript) for team speed.
- Strong structure for agency handoff (modular backend, typed contracts).
- Proven fit for marketplace workflows, file uploads, notifications, and reporting.

11. Non-Functional Requirements

11.1 Performance

- Mobile screens load target: <2s on Android 8.0 / 2GB RAM baseline.
- Web/admin dashboards target: <3s for large lists (10,000 records with pagination).

11.2 Reliability

- Offline-tolerant courier app:
 - cache assigned jobs.
 - queue user actions and sync on reconnect.

11.3 Security

- JWT auth with refresh strategy.
- Role and verification-state enforcement on every protected endpoint.
- Input validation and rate limiting.
- Audit logs for admin operations.

11.4 Data and Media Constraints

- Document upload max: 5MB raw input.
- Auto-compress image outputs for efficient mobile data use.
- GPS ping interval default: 30 minutes during active jobs.

11.5 Localization

- Courier app: French + Darija.
- Shipper/admin web: French MVP.

12. UX and Product Rules

12.1 Courier UX Rules

- Core actions should be achievable in 3 taps where possible.
- Large touch targets and simple language.
- Clear online/offline indicator.

12.2 Shipper UX Rules

- Dense list/table job management.
- Inline bid actions (accept/reject/counter) without deep navigation.

12.3 Shared Rules

- Always show fee and total math before confirmation.
- Use explicit status labels in every screen.

13. Acceptance Criteria for Delivery Agency

The implementation is accepted when all items below are true:

1. Full MVP flow works end-to-end for all 3 roles.
2. Role boundaries are enforced exactly as specified.
3. Verification process supports approve/reject with reasons.
4. Job lifecycle rules and transition validations are enforced server-side.
5. Bid handling supports accept/reject/counter and expiration.
6. POD upload is required before delivery completion.
7. Ratings and commission records are generated on close.
8. Admin can manage disputes and view audit logs.
9. Core performance targets are met under representative load.
10. Production-ready deployment includes monitoring, backups, and restore procedure.

14. Suggested Build Plan (Execution Order)

1. Monorepo/project setup and shared contracts.

2. Auth + RBAC + verification state model.
3. Core data schema and migrations.
4. Shipper job posting and courier load discovery.
5. Bidding and assignment engine.
6. Execution flow, GPS pinging, POD upload.
7. Chat and push notifications.
8. Ratings, commissions, and reporting.
9. Admin verification, disputes, moderation, audit tooling.
10. Hardening: performance, security, QA, deployment pipeline.

15. Final Notes to Development Team

- This file is authoritative for MVP.
- If conflicts appear with legacy docs, this file wins.
- Any deviation must be recorded as a change request with technical and business impact.